| | |
|---|---|
| **Mark** | **5/11** |

| | | | |
|---|---|---|---|
| **Team name:** | B9 | | |
| **Homework number:** | Homework 05 | | |
| **Due date:** | 21/10/2025 | | |
| **Contribution:** | **No** | **Partial** | **Full** |
| Pica Mirko | | | ✅ |
| Prendin Christian | | | ✅ |
| Pignataro Gabriele | | | ✅ |
| Moazenkakhki Mostafa | | | ✅ |
| Yarrabbotula Sunandha | | | ✅ |
| **Notes:** | | | |

| | | | |
|---|---|---|---|
| **Project name** | LCD & ADC | | |
| **Not done** | **Partially done (major problems)** | **Partially done (minor problems)** | **Completed** |
| | | | ✅ |

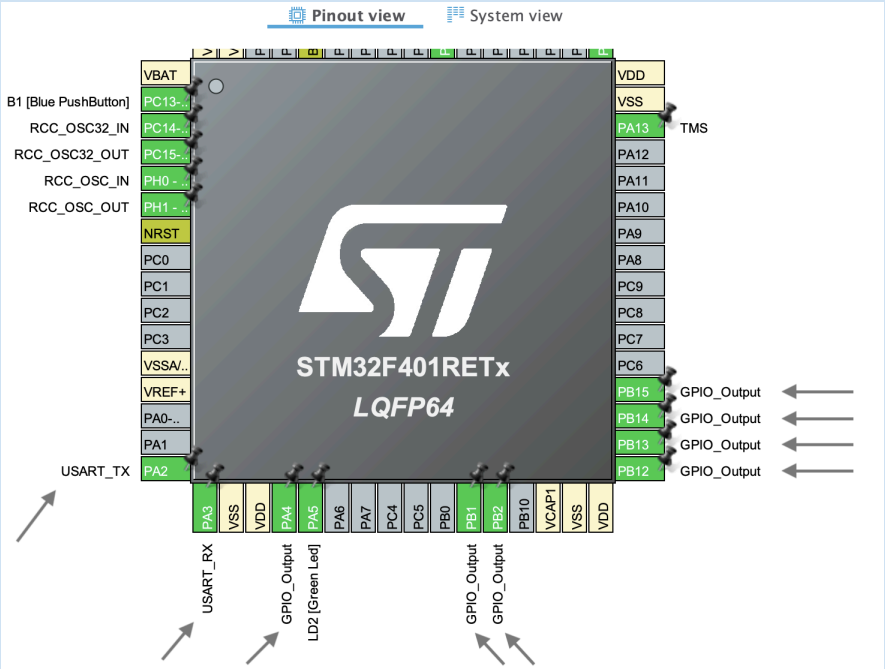
**Part 1. Try sending from the PC via UART a string of variable length that is displayed on the LCD. Hint: send one letter per time. If you are curious, there are also smarter solutions which are not so immediate.**

From the "Hands-on Lab Schematics.pdf" and the NUCLEO board schematics, we identified the necessary pins:

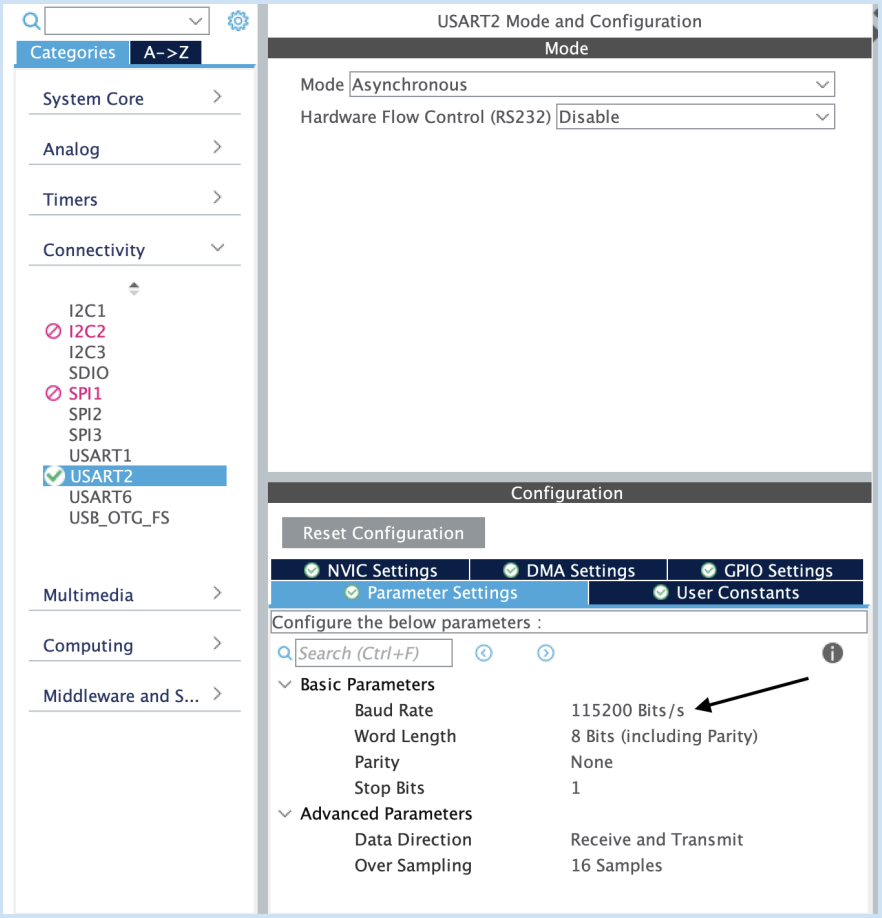
- USART2 (Virtual COM Port): **PA2** (TX) and **PA3** (RX).
- LCD Control: **PB1** (LCD_E) and **PB2** (LCD_RS).
- LCD Data (4-bit): **PB12** (D4), **PB13** (D5), **PB14** (D6), **PB15** (D7).
- LCD Backlight: **PA4** (LCD_BL_ON).

We configured the peripherals in STM32CubeIDE as follows:

1. **GPIO:**
    - **PA2** configured as USART2_TX.
    - **PA3** configured as USART2_RX.
    - **PB1, PB2, PB12, PB13, PB14, PB15** configured as GPIO_Output.
    - **PA4** configured as GPIO_Output.

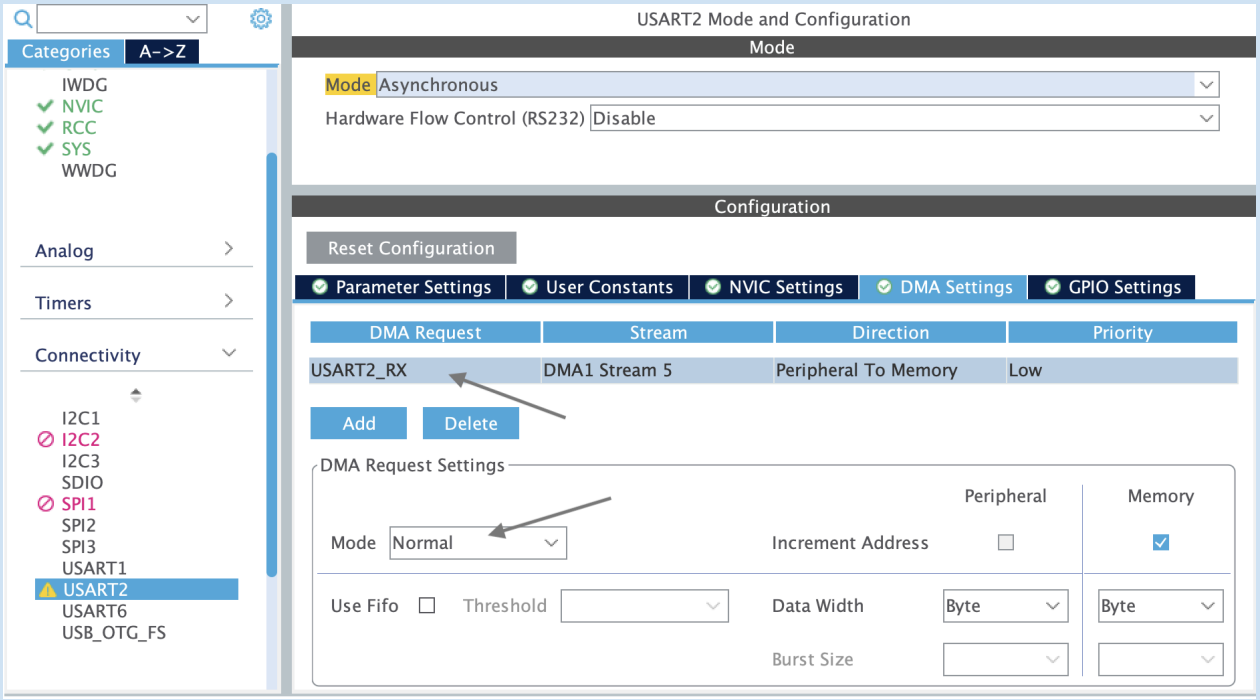



2. **USART:**
    - Mode set to **Asynchronous**.
    - Baud Rate set to **115200 Bits/s.**
    - Word Length set to **8 Bits** (including Parity).
    - Parity set to **None**.
    - Stop Bits set to **1**

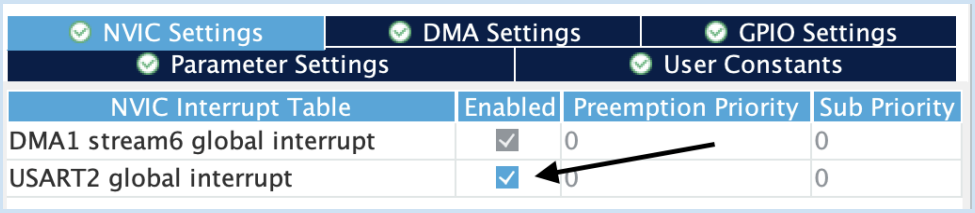



## 3. DMA:

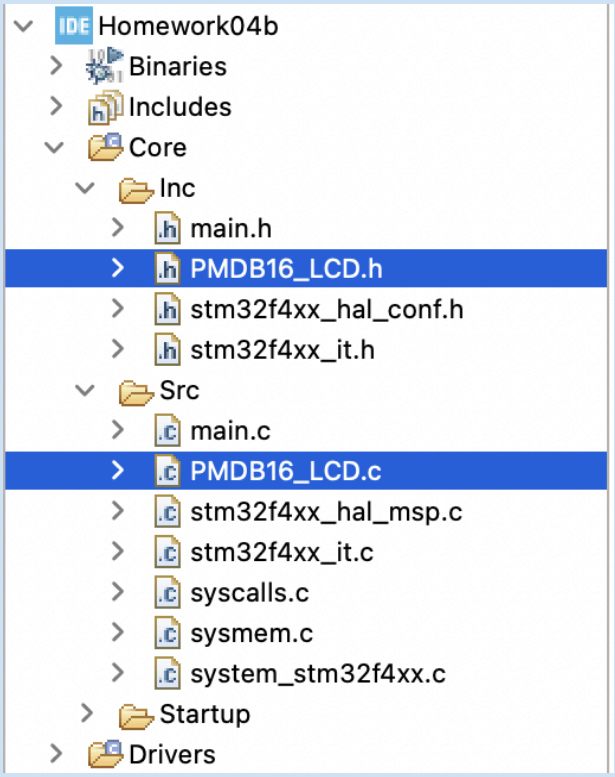
- Added a **DMA request** for USART2_RX using **DMA1 Stream 5**.
- Set DMA Mode to **Normal**.



4. **NVIC:**
   - Enabled the USART2 **global interrupt**.




To make the LCD work, we used the provided **PMDB16_LCD** library by importing the files "**PMDB16_LCD.c**" to the *project/Core/Source* folder and "**PMDB16_LCD.h**" to the *project/Core/Inc* folder, and in the end including it in our main.c




In the "main.c" file we make sure to include the necessary libraries:

```
/* Private includes ----------------------
/* USER CODE BEGIN Includes */
#include "PMDB16_LCD.h"
#include <string.h>
/* USER CODE END Includes */
```

Defined a receive buffer uint8_t rx_buffer[34]:

```
/* USER CODE BEGIN PV */
#define RX_BUFFER_SIZE 34
uint8_t rx_buffer[RX_BUFFER_SIZE];
/* USER CODE END PV */
```

Before the while loop, we initialize the LCD and started the DMA reception with Idle Line detection:

```c
  /* USER CODE BEGIN 2 */
  lcd_initialize();
  lcd_backlight_ON();
  lcd_clear();
  HAL_UARTEx_ReceiveToIdle_DMA(&huart2, rx_buffer, RX_BUFFER_SIZE);
  /* USER CODE END 2 */
```

In the end, we just implemented the HAL_UARTEx_RxEventCallback function:

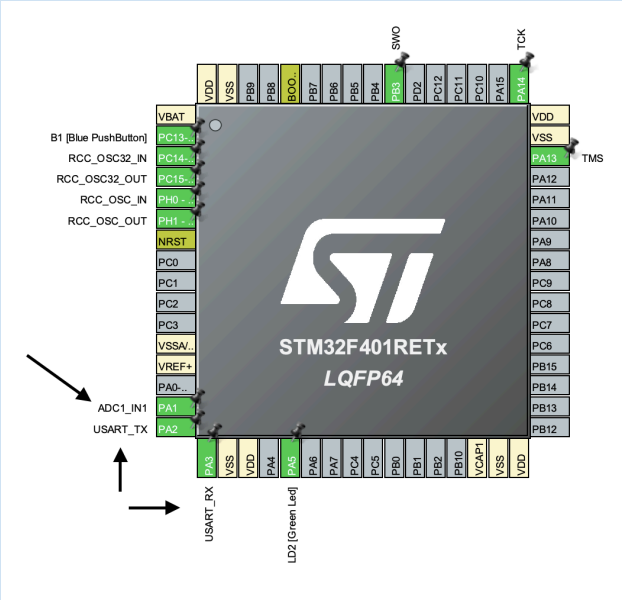
```c
/* USER CODE BEGIN 4 */
void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart, uint16_t Size)
{
  if (huart->Instance == USART2)
  {
    lcd_clear();

    // Case 1: The message fits on a single line (up to 16 chars)
    if (Size <= 16)
    {
      rx_buffer[Size] = '\0';            // Null-terminate the string
      lcd_println((char*)rx_buffer, 0); // Print on the first row
    }
    // Case 2: The message needs two lines (17 to 32 chars)
    else if (Size <= 32)
    {
      char line1[17];                    // 16 chars + null terminator
      char line2[17];

      // Copy the first 16 chars to line1
      strncpy(line1, (char*)rx_buffer, 16);
      line1[16] = '\0';

      // Copy the rest of the chars to line2
      strncpy(line2, (char*)rx_buffer + 16, Size - 16);
      line2[Size - 16] = '\0';

      lcd_println(line1, 0);             // Print first part on row 0
      lcd_println(line2, 1);             // Print second part on row 1
    }
    // Case 3: The message is too long (more than 32 chars)
    else
    {
      char line1[17];
      char line2[17];

      // Copy the first 16 chars to line1
      strncpy(line1, (char*)rx_buffer, 16);
      line1[16] = '\0';

      // Copy the next 14 chars to line2 to leave space for dots
      strncpy(line2, (char*)rx_buffer + 16, 14);

      // Add the two dots at the end of the second line
      line2[14] = '.';
      line2[15] = '.';
      line2[16] = '\0';                  // Null-terminate

      lcd_println(line1, 0);
      lcd_println(line2, 1);
    }

    // Re-arm the DMA reception for the next message
    HAL_UARTEx_ReceiveToIdle_DMA(&huart2, rx_buffer, RX_BUFFER_SIZE);
  }
}
/* USER CODE END 4 */
```

We tested the implementation using a serial terminal on the PC set to 115200 baud and configured to send a Newline character on Enter. The system successfully received strings of various lengths and displayed them correctly on the LCD according to our logic (single line, two lines, or truncated with ".."). The use of DMA with Idle Line detection proved robust against fast typing, resolving potential character loss issues. We have tested it and it works.

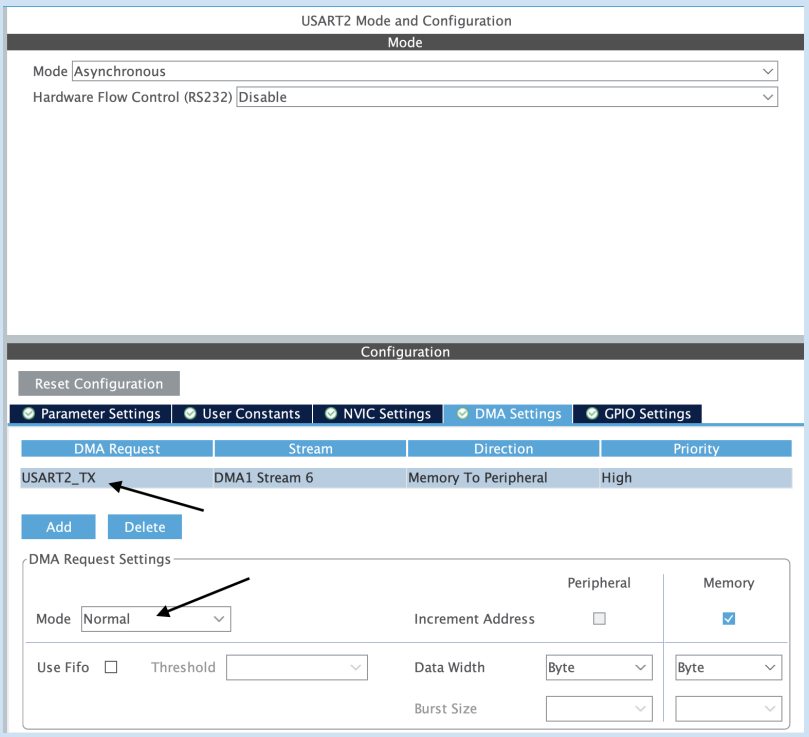
**2. Complete the ADC projects 2b and 2c.**

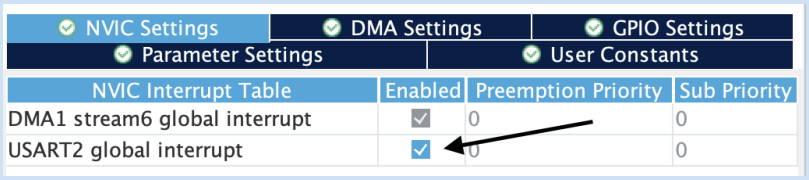
We found out from the “Hands-on Lab Schematics.pdf” file that the potentiometer is connected to pin PA1.

So we create a new project and we check if UART pins are enabled, and enable the potentiometer pin as ADC1_IN.

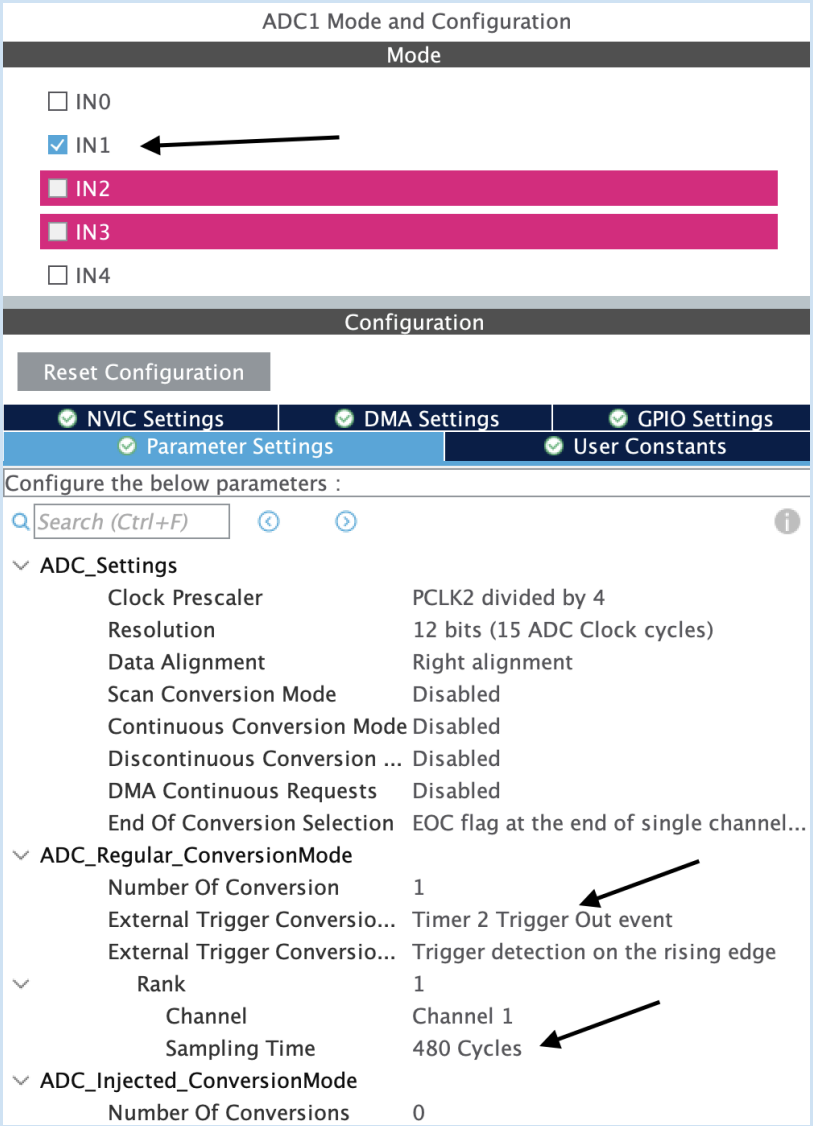


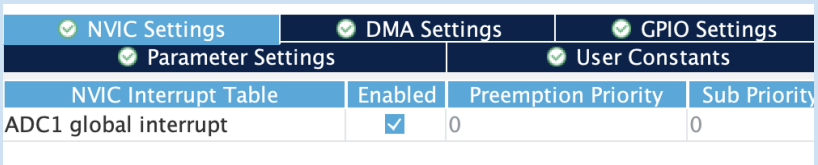

As in the previous homework we go into the connectivity section in USART2 and we enable the interrupt and the DMA in transmission.



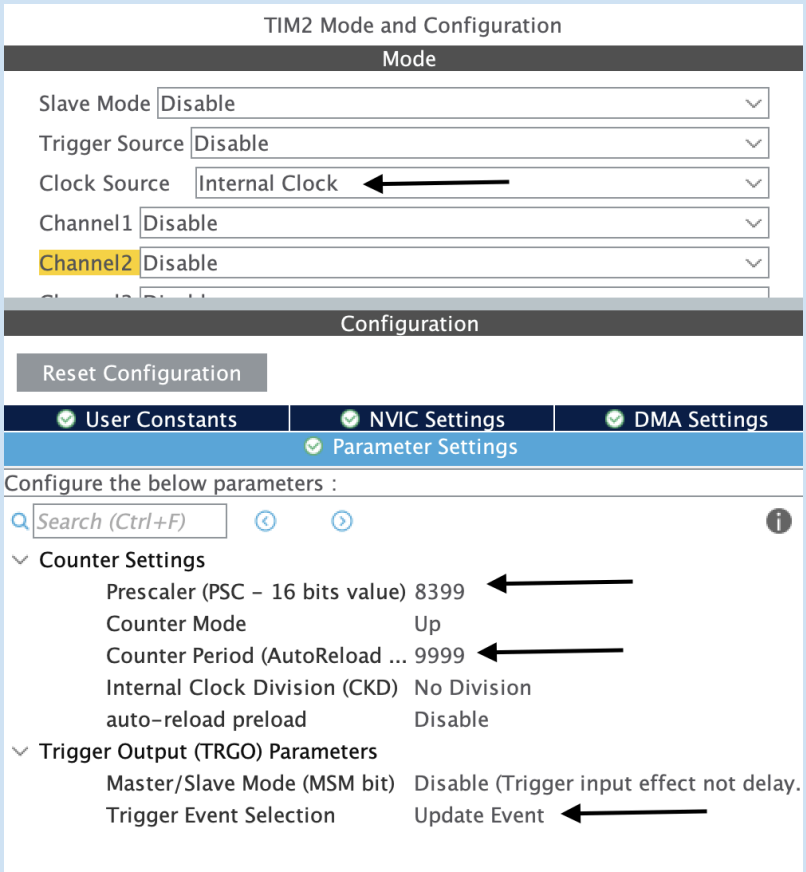



We now go into the analog section in ADC1, IN1 should be enabled (after enabling pin PA1). In the settings we leave everything default except for “External Trigger Conversion Source” where we put “Timer 2 trigger out event” and “Sampling Time” where we put “480 cycles”. And of course we enable the ADC1 global interrupt.




Now, we have to configure TIM2 to trigger every second. We set the clock source, the prescaler at 8399 and the counter at 9999. This time we do not need the timer interrupt, instead we want to trigger the ADC so in “Trigger event selection” we put “Update event”.



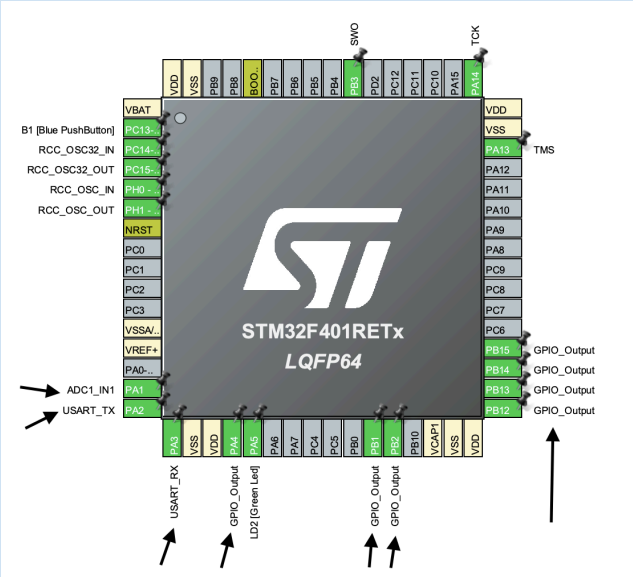

In the “main.c” file we just go into the main function and start both the adc and the timer.

```
/* USER CODE BEGIN 2 */
HAL_ADC_Start_IT(&hadc1);
HAL_TIM_Base_Start(&htim2);
/* USER CODE END 2 */
```

Then we write the ADC complete callback function as follows. (To get snprintf working, we need to import stdio.h and enable the float formatting in the project settings)

```
/* USER CODE BEGIN 0 */

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc) {
    int conversion = HAL_ADC_GetValue(&hadc1);
    float voltage = conversion*3.3/4096.0;
    char string[64];
    int length = snprintf(string, sizeof(string), "Voltage: %.3f V\n", voltage);
    HAL_UART_Transmit_DMA(&huart2, (uint8_t *)string, length);
}
/* USER CODE END 0 */
```

For project 2c it’s pretty much the same. We don’t need to configure the USART2 and the DMA, but we have to configure the pins for the LCD.




Import the needed library, and modify the “main.c” file.
In the main function we add the LCD initialization.

```
/* USER CODE BEGIN 2 */
lcd_initialize();
HAL_TIM_Base_Start(&htim2);
HAL_ADC_Start_IT(&hadc1);
/* USER CODE END 2 */
```

And in the conversion complete callback function we delete the part of the UART and add the code to print the voltage on the first row of the LCD and the corresponding bar on the second row of the LCD.

```
/* USER CODE BEGIN 0 */
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc) {
    int conversion = HAL_ADC_GetValue(&hadc1);
    float voltage = conversion*3.3/4096.0;
    char string[17];
    snprintf(string, sizeof(string), "Voltage: %.3f V", voltage);
    lcd_println(string, 0);
    int bar = voltage * 80 / 3.3;
    lcd_drawBar(bar);
}
/* USER CODE END 0 */
```

| **Professor comments:** | |
|---|---|